

# BATL Coding Guidelines

## **Table of Contents:**

|                                                   |    |
|---------------------------------------------------|----|
| 1. Introduction: .....                            | 5  |
| 2. Targeted Audience:.....                        | 5  |
| 3. Variable Naming Guidelines: .....              | 5  |
| 3.1 Naming Pattern: .....                         | 5  |
| 3.2 Components of the Pattern:.....               | 5  |
| 3.2.1 Type of variable: .....                     | 5  |
| 3.2.2 Scope indicator:.....                       | 5  |
| 3.2.3 Type indicator: .....                       | 5  |
| 3.2.4 Class indicator:.....                       | 6  |
| 3.2.5 Location indicator: .....                   | 6  |
| 3.2.6 Name of variable:.....                      | 6  |
| 3.3 Examples:.....                                | 6  |
| 4. Function Naming Guidelines:.....               | 8  |
| 4.1 Naming pattern: .....                         | 8  |
| 4.2 Components of the pattern: .....              | 8  |
| 4.2.1 Scope indicator:.....                       | 8  |
| 4.2.2 Return type indicator:.....                 | 8  |
| 4.2.3 Location indicator: .....                   | 8  |
| 4.2.4 Name of function: .....                     | 8  |
| 4.3 Examples:.....                                | 9  |
| 5. User-Defined Data Type Naming Guidelines:..... | 10 |
| 5.1 General rules: .....                          | 10 |
| 5.2 Naming pattern: .....                         | 10 |
| 5.3 Components of the pattern: .....              | 10 |
| 5.3.1 Type: .....                                 | 10 |
| 5.3.2 Name: .....                                 | 10 |
| 5.3.3 Suffix: .....                               | 10 |
| 5.4 Examples:.....                                | 10 |
| 6. Enum Member Naming Guidelines:.....            | 11 |
| 6.1 Naming pattern: .....                         | 11 |
| 6.2 Examples:.....                                | 11 |
| 7. Change log:.....                               | 12 |

**List of Tables:**

|                                                      |    |
|------------------------------------------------------|----|
| Table 1: Scope indicator .....                       | 5  |
| Table 2: Type indicator.....                         | 6  |
| Table 3: Class indicator .....                       | 6  |
| Table 4: Variable naming examples .....              | 7  |
| Table 5: Scope indicator .....                       | 8  |
| Table 6: Function naming examples.....               | 9  |
| Table 7: Suffix for user-defined data types.....     | 10 |
| Table 8: User-defined data type naming examples..... | 10 |

## BATL Coding Guideline Documentation

### Functional Description:

*This document outlines the guidelines for documenting firmware within your codebase. Proper code documentation enhances code readability, maintainability and facilitates easier collaboration among developers.*

### Document Control:

|                                                      |
|------------------------------------------------------|
| IMPORTANT NOTICE<br>Confidentiality & Restricted Use |
|------------------------------------------------------|

- *Intended Recipient(s): Bajaj Auto Technology Limited – Intelligent Systems Group – EEE Department, Corp. R&D.*
- *Name:*
- *The above-named recipient(s) is/are responsible for the safe keeping of this document and ensuring that the document and information contained in it are used only in accordance with the agreements with Bajaj Auto Ltd., hereinafter referred to as BATL.*
- *This document and the information contained within it are being provided “for information purposes only” and BATL shall not be held liable as to the accuracy or completeness of the information.*
- *All Intellectual Property Rights, including but not limited to Copyright, applying to this document and the information contained therein vest solely and exclusively with BATL. No part of this document may be copied or reproduced, either in part or in full by any means, whether mechanical or electronic, without prior written permission of the authorized signatory of BATL. BATL reserves all rights to deal with violations of this clause in accordance with applicable laws.*
- *Due to the confidential nature of the information in this document, internal dissemination/copying should be made on a “need to know” basis only and this “Document Control” notice should be included with all copies.*
- *This document remains the property of BATL.*

## 1. Introduction:

This document outlines the guidelines for documenting firmware within the codebase. Proper code documentation enhances code readability, maintainability and facilitates easier collaboration among developers. It covers guidelines for declaring variables, functions and user-defined variables. Additionally, it also proposes naming guidelines for enum members.

## 2. Targeted Audience:

This document is intended to be shared with BATL Intelligent System group developers.

## 3. Variable Naming Guidelines:

Variable names should be concise yet descriptive enough to convey the purpose of the variable.

### 3.1 Naming Pattern:

Follow a consistent pattern for variable names to enhance readability and predictability. The proposed naming pattern is as follows:

*<Type of variable>< Scope indicator><Type indicator><Class indicator><\_><Location indicator><\_><Name of variable>*

### 3.2 Components of the Pattern:

#### 3.2.1 Type of variable:

Indicates the data type or purpose of the variable

#### 3.2.2 Scope indicator:

Specifies the scope of the variable (e.g., local, global).

| Scope        | Scope indicator |
|--------------|-----------------|
| Global       | g               |
| Static       | s               |
| Local        | Not Applicable  |
| Static Local | sl              |

Table 1: Scope indicator

#### 3.2.3 Type indicator:

Indicates the data type (e.g., uint8\_t, sint32\_t, float).

| Type     | Type indicator |
|----------|----------------|
| uint8_t  | u8             |
| uint16_t | u16            |
| uint32_t | u32            |
| uint64_t | u64            |

|                |     |
|----------------|-----|
| int8_t         | s8  |
| int16_t        | s16 |
| int32_t        | s32 |
| int64_t        | s64 |
| char           | c   |
| bool           | b   |
| single         | f16 |
| double         | f32 |
| SampleEnum_E   | e   |
| SampleStruct_T | st  |
| SampleUnion_U  | u   |
| funcPtr        | fpt |
| void           | v   |

*Table 2: Type indicator*

### 3.2.4 Class indicator:

Denotes the classification of the variable (e.g., array, pointer, structure)

| Class   | Class indicator |
|---------|-----------------|
| Pointer | pt              |
| Array   | ar              |

*Table 3: Class indicator*

### 3.2.5 Location indicator:

- Module name where the variable is defined, written in block letters (e.g. BLK, FSM, IPC), followed by an \_.
- Mandatory for global, static and static local variables. Several modules often hold variables with the same purpose (e.g. an initialisation flag); the location indicator tells them apart, e.g. sb\_BLK\_initialized instead of sb\_initialized.
- Not applicable for local variables.

### 3.2.6 Name of variable:

- The data type qualifier string is followed by an \_.
- The variable name should be written in Lower Camel Case, i.e. it begins with a lowercase letter, also when it follows the location indicator (e.g. sst\_BLK\_lock, not sst\_BLK\_Lock).
- Function names begin in Pascal Case (refer section 4.2.4), so the case of the first letter of the name distinguishes a variable from a function, e.g. sst\_BLK\_cfg is a variable whereas gi\_BLK\_Init is a function.
- The name must always begin with an alphabet.

## 3.3 Examples:

| Variable declaration                                   | Description                                                                                                         |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>static uint8_t su8ar_FSM_sampleArray[10];</code> | Private array containing uint8_t type elements, residing in FSM module                                              |
| <code>SampleStruct_T gst_FSM_sampleStruct;</code>      | Public structure, residing in FSM module                                                                            |
| <code>uint16_t *gu16pt_FSM_sampleEnum;</code>          | Public pointer to uint16_t, residing in FSM module                                                                  |
| <code>SampleEnum_E e_sampleEnum;</code>                | Local enum (location indicator not applicable)                                                                      |
| <code>SampleUnion_U gu_FSM_sampleUnion;</code>         | Public union, residing in FSM module                                                                                |
| <code>SampleFuncPtr_F *gfpt_FSM_sampleCallBack</code>  | Global function pointer, note return type of function and parameters to function need not to be specified in naming |
| <code>static UInt16_T slu16_FSM_sampleVar = 0;</code>  | Static variable local to a function, residing in FSM module.                                                        |

Table 4: Variable naming examples

## 4. Function Naming Guidelines:

All the functions should be declared with below mentioned guidelines:

### 4.1 Naming pattern:

Follow a consistent pattern for function names to enhance readability and predictability. The proposed naming pattern is as follows:

*<Scope indicator><Return type indicator>\_<Location indicator>\_<Name of function>*

### 4.2 Components of the pattern:

#### 4.2.1 Scope indicator:

Specifies the scope of the variable (e.g., local, global).

| Scope  | Scope indicator |
|--------|-----------------|
| Global | g               |
| Static | s               |
| Local  | Not Applicable  |

*Table 5: Scope indicator*

#### 4.2.2 Return type indicator:

Same as variable, the return value of a function can be mentioned at the beginning of function name.

#### 4.2.3 Location indicator:

- Module name where function is defined should also be preferably included in the function name.
- So, for example, it will be easy to identify LIB, LIL, MCAL which hosts the function. As per the location of the function it can be FSM, IPC, BNET, SEC, etc.
- The location naming is only applicable for global functions and private/local functions may not include this field.

#### 4.2.4 Name of function:

- The name of the function should begin in Pascal Case (each word starts with a capital letter), please refer to example below.
- Also please do note that the **return type indicator field** and **location indicator field** should be followed by an underscore '\_'.



**4.3 Examples:**

| Function declaration         | Description                                            |
|------------------------------|--------------------------------------------------------|
| uint8_t su8_SampleFun1();    | Private function returning uint8_t type element        |
| SampleEnum_E e_SampleFun2(); | Local function returning SampleEnum_E                  |
| void gv_FSM_SampleFun3();    | Public function returning void, residing in FSM module |

*Table 6: Function naming examples*

## 5. User-Defined Data Type Naming Guidelines:

The following guidelines establish a standard approach for naming user-defined data types in your codebase. Consistency in naming user-defined data types enhances code readability, maintainability, and collaboration among developers.

### 5.1 General rules:

### 5.2 Naming pattern:

User-defined data type names should be descriptive enough to convey the purpose. Follow a consistent pattern for user-defined data type names to enhance readability and predictability. The proposed naming pattern is as follows:

**<Type><Name><Suffix>**

### 5.3 Components of the pattern:

#### 5.3.1 Type:

Specifies the kind of data type being defined (e.g., struct, enum, union, typedef).

#### 5.3.2 Name:

- Written in Pascal Case (each word starts with a capital letter)
- Should clearly describe the purpose or structure of the data type.

#### 5.3.3 Suffix:

| Data type        | Suffix |
|------------------|--------|
| Structure        | _T     |
| Union            | _U     |
| Enum             | _E     |
| Function pointer | _F     |

Table 7: Suffix for user-defined data types

### 5.4 Examples:

| Data type declaration                   | Description                                         |
|-----------------------------------------|-----------------------------------------------------|
| typedef struct {...} SampleStruct_T;    | Typedef for structure named SampleStruct            |
| typedef struct {...} SampleEnum_E;      | Typedef for enum named SampleEnum                   |
| typedef struct {...} SampleUnion_U;     | Typedef for union named SampleUnion                 |
| typedef void (*SampleCallBack_F)(void); | Typedef for a function pointer named SampleCallBack |

Table 8: User-defined data type naming examples

## 6. Enum Member Naming Guidelines:

The following guidelines establish a standard approach for naming enum members. It will also help developers to distinguish between enums and defines as they both usually are written in block letters. Consistency in naming user-defined data types enhances code readability, maintainability, and collaboration among developers.

### 6.1 Naming pattern:

Enum member names should be descriptive enough to convey the purpose. Follow a consistent pattern for to enhance readability and predictability. The proposed naming pattern is as follows:

*<e><Acronym of enum data type>\_<Name of member>*

- Name should be in block letters.
- Use ‘\_’ to separate words as there are all the block letters.
- To distinguish similar enum members from other enum data type, mention acronym of enum data type.

### 6.2 Examples:

```
typedef enum
{
    eSE_MEMBER_1,
    eSE_MEMBER_2,
} SampleEnum_E;
```

## 7. Change log:

| Version | Data of change | Modified by      | Summary                                                                                                                                                                                                                                                                                                                            | Section modified                 |
|---------|----------------|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| 1.0     | 01/08/2024     | SHIVAM CHUDASAMA | First draft.                                                                                                                                                                                                                                                                                                                       | All                              |
| 1.1     | 27/03/2025     | YASH GIRAMKAR    | <ul style="list-style-type: none"> <li>- Changes incorporated to include guidelines for creation of static local variables.</li> <li>- Example of the same is also included in corresponding section.</li> </ul>                                                                                                                   | -3.2.2<br>-3.3                   |
| 1.2     | 23/09/2026     | SHIVAM CHUDASAMA | <ul style="list-style-type: none"> <li>- Location indicator (module name) added to the variable naming pattern for global, static and static local variables.</li> <li>- Clarified that the variable name after the location indicator begins in lower case, to distinguish variables from functions. Examples updated.</li> </ul> | -3.1<br>-3.2.5<br>-3.2.6<br>-3.3 |